

LeetCode 75

Detailed Solutions Guide

Complete Explanations with Visual Illustrations

Each problem includes step-by-step solutions, visual diagrams, complexity analysis, and coding implementations

Array / String Problems

Problem 1: Merge Strings Alternately

Difficulty: Easy

Problem Statement:

You are given two strings word1 and word2. Merge the strings by adding letters in alternating order, starting with word1. If a string is longer than the other, append the additional letters onto the end of the merged string.

Example:

Input: word1 = 'abc', word2 = 'pqr'

Output: 'apbqcr'

Explanation: The merged string will be: a + p + b + q + c + r

Approach:

Use two pointers to iterate through both strings simultaneously. At each step, append the current character from each string (if available) to the result. When one string is exhausted, append the remaining characters from the other string.

Step-by-Step Solution:

Step	word1[i]	word2[j]	Result
1	a (i=0)	p (j=0)	'ap'
2	b (i=1)	q (j=1)	'apbq'
3	c (i=2)	r (j=2)	'apbqcr'

Python Solution:

```
def mergeAlternately(word1, word2):
    result = []
    i, j = 0, 0

    # Alternate between word1 and word2
    while i < len(word1) and j < len(word2):
        result.append(word1[i])
        result.append(word2[j])
        i += 1
        j += 1

    # Append remaining characters
    result.append(word1[i:])
    result.append(word2[j:])

    return ''.join(result)
```

Complexity Analysis:

Time Complexity: $O(n + m)$ where n and m are lengths of word1 and word2

Space Complexity: $O(n + m)$ for the result string

Key Insights:

This is a straightforward two-pointer problem. The key is handling the remaining characters after one string is exhausted. Using string slicing (`word[i:]`) elegantly handles both the case where characters remain and where the string is fully consumed.

Problem 2: Greatest Common Divisor of Strings

Difficulty: Easy

Problem Statement:

For two strings s and t , we say that t divides s if and only if $s = t + t + \dots + t$ (i.e., t is concatenated with itself one or more times). Given two strings $str1$ and $str2$, return the largest string x such that x divides both $str1$ and $str2$.

Example:

```
Input: str1 = 'ABCABC', str2 = 'ABC'
```

```
Output: 'ABC'
```

Explanation: 'ABC' divides both 'ABCABC' (ABC + ABC) and 'ABC' (ABC)

Approach:

First, check if a GCD string exists by verifying that $str1 + str2$ equals $str2 + str1$. If not, no common divisor exists. If they match, the GCD string has length equal to $\text{gcd}(\text{len}(str1), \text{len}(str2))$. Extract the substring of that length from either string.

Visual Example:

```
str1 = 'ABCABC' (length 6)
```

```
str2 = 'ABC' (length 3)
```

```
gcd(6, 3) = 3
```

```
str1 + str2 = 'ABCABCABC'
```

```
str2 + str1 = 'ABCABCABC' ✓ (same)
```

```
Result: str1[0:3] = 'ABC'
```

Python Solution:

```
from math import gcd

def gcdOfStrings(str1, str2):
    # Check if GCD string exists
    if str1 + str2 != str2 + str1:
        return ""

    # Get GCD of lengths
    gcd_len = gcd(len(str1), len(str2))

    # Return the GCD string
    return str1[:gcd_len]
```

Complexity Analysis:

Time Complexity: $O(n + m)$ for string concatenation comparison

Space Complexity: $O(n + m)$ for concatenated strings

Key Insights:

The critical observation is that if a GCD string exists, then $\text{str1} + \text{str2}$ must equal $\text{str2} + \text{str1}$. This is because both strings are composed of repetitions of the same base string. The mathematical GCD of the lengths gives us the exact length of the base repeating unit.

Problem 3: Kids With the Greatest Number of Candies

Difficulty: Easy

Problem Statement:

There are n kids with candies. You are given an integer array `candies`, where each `candies[i]` represents the number of candies the i th kid has, and an integer `extraCandies`. Return a boolean array `result` of length n , where `result[i]` is `true` if, after giving the i th kid all the `extraCandies`, they will have the greatest number of candies among all the kids, or `false` otherwise.

Example:

Input: `candies = [2,3,5,1,3]`, `extraCandies = 3`

Output: `[true,true,true,false,true]`

2	3	5	1	3
0	1	2	3	4

Explanation:

Kid	Current	+ Extra	Total	Max Needed	Result
0	2	3	5	5	True
1	3	3	6	5	True
2	5	3	8	5	True
3	1	3	4	5	False
4	3	3	6	5	True

Approach:

First, find the maximum number of candies any kid currently has. Then, for each kid, check if their candies plus `extraCandies` is greater than or equal to this maximum. This gives us $O(n)$ time with a single pass to find max and another to build the result.

Python Solution:

```
def kidsWithCandies(candies, extraCandies):
    max_candies = max(candies)
    result = []

    for candy in candies:
        result.append(candy + extraCandies >= max_candies)

    return result

# Or more concisely:
def kidsWithCandies(candies, extraCandies):
```

```
max_candies = max(candies)
return [candy + extraCandies >= max_candies for candy in candies]
```

Complexity Analysis:

Time Complexity: $O(n)$ - one pass to find max, one to build result

Space Complexity: $O(n)$ for the result array

Problem 4: Can Place Flowers

Difficulty: Easy

Problem Statement:

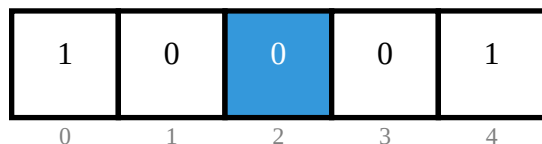
You have a long flowerbed in which some plots are planted and some are not. Flowers cannot be planted in adjacent plots. Given an integer array `flowerbed` containing 0s and 1s (0 means empty, 1 means planted) and an integer `n`, return `true` if `n` new flowers can be planted without violating the no-adjacent-flowers rule.

Example:

Input: `flowerbed = [1,0,0,0,1]`, `n = 1`

Output: `true`

Visual Representation:



Index 2 (highlighted) can have a flower planted

Approach:

Iterate through the flowerbed. For each empty plot (0), check if both adjacent plots are also empty (or don't exist at boundaries). If so, plant a flower and increment the count. Continue until we've planted `n` flowers or exhausted all possibilities.

Step-by-Step Example:

`flowerbed = [1,0,0,0,1]`, `n = 1`

Index	Value	Left	Right	Can Plant?	Action
0	1	N/A	0	No	Skip (occupied)
1	0	1	0	No	Skip (left occupied)
2	0	0	0	Yes	Plant! count=1
3	0	1	1	No	Skip (both occupied)
4	1	0	N/A	No	Skip (occupied)

Python Solution:

```
def canPlaceFlowers(flowerbed, n):
    count = 0

    for i in range(len(flowerbed)):
        # Check if current position is empty
        if flowerbed[i] == 0:
            # Check left neighbor (or treat as empty if at boundary)
```



```
empty_left = (i == 0) or (flowerbed[i-1] == 0)
# Check right neighbor (or treat as empty if at boundary)
empty_right = (i == len(flowerbed)-1) or (flowerbed[i+1] == 0)

# If both neighbors are empty, plant a flower
if empty_left and empty_right:
    flowerbed[i] = 1
    count += 1

# Early termination
if count >= n:
    return True

return count >= n
```

Complexity Analysis:

Time Complexity: $O(m)$ where m is length of flowerbed

Space Complexity: $O(1)$ - modifying input in-place

Key Insights:

The greedy approach works here because planting a flower as soon as possible never prevents us from planting more flowers later. Handle boundary conditions carefully by treating positions before index 0 and after the last index as empty. Early termination when count reaches n optimizes the solution.

Problem 5: Reverse Vowels of a String

Difficulty: Easy

Problem Statement:

Given a string *s*, reverse only all the vowels in the string and return it. The vowels are 'a', 'e', 'i', 'o', and 'u', and they can appear in both lower and upper cases, more than once.

Example:

Input: *s* = 'hello'

Output: 'holle'

Visual Process:

Step	String State	Left Pointer	Right Pointer	Action
Initial	h e l l o	e (1)	o (4)	Both vowels
1	h o l l e	-	-	Swap e and o
Result	holle	-	-	Done

Approach:

Use the two-pointer technique. Start with left pointer at the beginning and right pointer at the end. Move left pointer forward until it finds a vowel. Move right pointer backward until it finds a vowel. Swap them and continue until pointers meet.

Python Solution:

```
def reverseVowels(s):
    vowels = set('aeiouAEIOU')
    s_list = list(s) # Convert to list for mutability
    left, right = 0, len(s) - 1

    while left < right:
        # Move left pointer to next vowel
        while left < right and s_list[left] not in vowels:
            left += 1

        # Move right pointer to previous vowel
        while left < right and s_list[right] not in vowels:
            right -= 1

        # Swap vowels
        s_list[left], s_list[right] = s_list[right], s_list[left]
        left += 1
        right -= 1

    return ''.join(s_list)
```

Complexity Analysis:

Time Complexity: $O(n)$ - single pass through the string

Space Complexity: $O(n)$ - converting string to list

Key Insights:

The two-pointer approach is optimal because each character is visited at most once. Using a set for vowel lookup provides $O(1)$ check time. Converting to a list is necessary in Python since strings are immutable. The inner while loops ensure we only swap when both pointers are at vowels.

Problem 6: Reverse Words in a String

Difficulty: Medium

Problem Statement:

Given an input string *s*, reverse the order of the words. A word is defined as a sequence of non-space characters. The words in *s* will be separated by at least one space. Return a string of the words in reverse order concatenated by a single space.

Example:

Input: *s* = 'the sky is blue'

Output: 'blue is sky the'

Visual Process:

Step	Action	Result
1	Split by spaces	['the', 'sky', 'is', 'blue']
2	Reverse array	['blue', 'is', 'sky', 'the']
3	Join with space	'blue is sky the'

Python Solution:

```
def reverseWords(s):
    # Method 1: Using built-in functions
    return ' '.join(s.split()[::-1])

# Method 2: Manual approach
def reverseWords(s):
    words = []
    word = []

    for char in s:
        if char != ' ':
            word.append(char)
        elif word: # End of a word
            words.append(''.join(word))
            word = []

    # Don't forget last word
    if word:
        words.append(''.join(word))

    # Reverse and join
    return ' '.join(reversed(words))
```

Complexity Analysis:

Time Complexity: $O(n)$ - process each character once

Space Complexity: $O(n)$ - store words and result

Problem 7: Product of Array Except Self

Difficulty: Medium

Problem Statement:

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`. You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example:

Input: `nums = [1,2,3,4]`

Output: `[24,12,8,6]`

Visualization - Prefix and Suffix Products:

Index	Value	Left Product	Right Product	Result
0	1	1	$2 \times 3 \times 4 = 24$	$1 \times 24 = 24$
1	2	1	$3 \times 4 = 12$	$1 \times 12 = 12$
2	3	$1 \times 2 = 2$	4	$2 \times 4 = 8$
3	4	$1 \times 2 \times 3 = 6$	1	$6 \times 1 = 6$

Approach:

Use two passes. First pass: compute products of all elements to the left of each index and store in result array. Second pass: multiply each result by the product of all elements to the right. This achieves $O(n)$ time without division.

Python Solution:

```
def productExceptSelf(nums):
    n = len(nums)
    result = [1] * n

    # First pass: calculate left products
    left_product = 1
    for i in range(n):
        result[i] = left_product
        left_product *= nums[i]

    # Second pass: multiply by right products
    right_product = 1
    for i in range(n-1, -1, -1):
        result[i] *= right_product
        right_product *= nums[i]

    return result
```

Complexity Analysis:

Time Complexity: $O(n)$ - two passes through array

Space Complexity: $O(1)$ - output array doesn't count

Problem 8: Container With Most Water

Difficulty: Medium

Problem Statement:

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are `(i, 0)` and `(i, height[i])`. Find two lines that together with the x-axis form a container that holds the most water. Return the maximum amount of water a container can store.

Example:

Input: `height = [1,8,6,2,5,4,8,3,7]`

Output: 49

Explanation: Lines at index 1 (height 8) and index 8 (height 7) form container with area = $7 \times 7 = 49$

Visual Explanation:

$\text{Area} = \min(\text{height}[\text{left}], \text{height}[\text{right}]) \times (\text{right} - \text{left})$

Left	Right	Height[L]	Height[R]	Width	Area	Max
0	8	1	7	8	$1 \times 8 = 8$	8
1	8	8	7	7	$7 \times 7 = 49$	49
1	7	8	3	6	$3 \times 6 = 18$	49
2	7	6	3	5	$3 \times 5 = 15$	49

Approach - Two Pointers (Greedy):

Start with widest container (`left=0`, `right=n-1`). The area is limited by the shorter height. Calculate current area. Move the pointer with the shorter height inward, as this is the only way to potentially find a larger area. Continue until pointers meet.

Why Move Shorter Height?

Moving the taller height pointer would only decrease both width and height (limited by shorter side), guaranteeing a smaller area. Moving the shorter height pointer maintains hope of finding a taller line that could compensate for the reduced width.

Python Solution:

```
def maxArea(height):
    left, right = 0, len(height) - 1
    max_area = 0

    while left < right:
        # Calculate current area
        width = right - left
        current_height = min(height[left], height[right])
        current_area = width * current_height
        max_area = max(max_area, current_area)
```

```
# Move pointer with shorter height
if height[left] < height[right]:
    left += 1
else:
    right -= 1

return max_area
```

Complexity Analysis:

Time Complexity: $O(n)$ - single pass with two pointers

Space Complexity: $O(1)$ - only using variables

Problem 9: Max Consecutive Ones III

Difficulty: Medium

Problem Statement:

Given a binary array `nums` and an integer `k`, return the maximum number of consecutive 1's in the array if you can flip at most `k` 0's.

Example:

Input: `nums = [1,1,1,0,0,1,1,1,1,0]`, `k = 2`

Output: 6

Explanation: Flip 0s at indices 4 and 5 to get `[1,1,1,0,0,1,1,1,1,1]` with 6 consecutive 1s from index 5-10

Sliding Window Approach:

Step	Window	Zeros	Valid?	Length	Max
1	[1,1,1]	0	Yes	3	3
2	[1,1,1,0]	1	Yes	4	4
3	[1,1,1,0,0]	2	Yes	5	5
4	[1,1,1,0,0,0]	3	No	-	5
5	[1,0,0,0]	3	No	Shrink left	5
6	[0,0,0,1]	3	No	Shrink left	5
7	[0,0,1,1]	2	Yes	4	5
8	[0,0,1,1,1]	2	Yes	5	5
9	[0,0,1,1,1,1]	2	Yes	6	6

Approach:

Use a sliding window with two pointers. Expand the window by moving right pointer. Count zeros in the window. When zeros exceed `k`, shrink from left until zeros equals `k` again. Track the maximum window size throughout.

Python Solution:

```
def longestOnes(nums, k):
    left = 0
    zeros = 0
    max_length = 0

    for right in range(len(nums)):
        # Expand window
        if nums[right] == 0:
            zeros += 1

        # Shrink window if too many zeros
        while zeros > k:
            if nums[left] == 0:
                zeros -= 1
            left += 1

        max_length = max(max_length, right - left + 1)
```

```
        left += 1

    # Update max length
    max_length = max(max_length, right - left + 1)

return max_length
```

Complexity Analysis:

Time Complexity: $O(n)$ - each element visited at most twice

Space Complexity: $O(1)$ - only using counters

Problem 10: Maximum Depth of Binary Tree

Difficulty: Easy

Problem Statement:

Given the root of a binary tree, return its maximum depth. A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example:

```
Input: root = [3,9,20,null,null,15,7]
```

```
Output: 3
```

Tree Visualization:

```
3 (depth 1)
```

```
/ \
```

```
9 20 (depth 2)
```

```
/ \
```

```
15 7 (depth 3)
```

Recursive Approach:

The depth of a tree is 1 + maximum depth of its subtrees. Base case: empty tree has depth 0. Recursively calculate depth of left and right subtrees, return 1 plus the maximum.

Recursion Tree:

Node	Left Depth	Right Depth	Result
3	1	2	$1 + \max(1,2) = 3$
9	0	0	$1 + \max(0,0) = 1$
20	1	1	$1 + \max(1,1) = 2$
15	0	0	1
7	0	0	1

Python Solution:

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def maxDepth(root):
    # Base case: empty tree
    if not root:
```

```

        return 0

    # Recursive case: 1 + max of subtree depths
    left_depth = maxDepth(root.left)
    right_depth = maxDepth(root.right)

    return 1 + max(left_depth, right_depth)

# Iterative BFS solution:
from collections import deque

def maxDepth(root):
    if not root:
        return 0

    queue = deque([root])
    depth = 0

    while queue:
        depth += 1
        # Process all nodes at current level
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

    return depth

```

Complexity Analysis:

Time Complexity: $O(n)$ - visit each node once

Space Complexity: $O(h)$ for recursion stack where h is height, $O(n)$ worst case

Problem 11: House Robber

Difficulty: Medium

Problem Statement:

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses are arranged in a line, and adjacent houses have security systems connected that will alert police if two adjacent houses are broken into on the same night. Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob without alerting the police.

Example:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and house 3 (money = 3). Total = 1 + 3 = 4

DP State Transition:

House	Value	Rob Current?	Don't Rob	Rob	Max
0	1	1	0	1	1
1	2	<code>prev_no_rob + 2</code>	1	2	2
2	3	<code>prev_no_rob + 3</code>	2	4	4
3	1	<code>prev_no_rob + 1</code>	4	3	4

Approach - Dynamic Programming:

At each house, we have two choices: rob it (gaining its value but can't rob previous house) or skip it (keeping maximum from previous house). The recurrence relation is: $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$. This can be optimized to use only two variables instead of an array.

Python Solution:

```
def rob(nums):
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]

    # dp[i] represents max money robbed up to house i
    prev2 = 0 # dp[i-2]
    prev1 = 0 # dp[i-1]

    for num in nums:
        # Either rob current + dp[i-2], or skip current (dp[i-1])
        current = max(prev1, prev2 + num)
        prev2 = prev1
        prev1 = current

    return prev1
```

```
# Alternative with DP array (easier to understand):
def rob(nums):
    if not nums:
        return 0
    if len(nums) == 1:
        return nums[0]

    dp = [0] * len(nums)
    dp[0] = nums[0]
    dp[1] = max(nums[0], nums[1])

    for i in range(2, len(nums)):
        dp[i] = max(dp[i-1], dp[i-2] + nums[i])

    return dp[-1]
```

Complexity Analysis:

Time Complexity: $O(n)$ - single pass through houses

Space Complexity: $O(1)$ with optimized solution, $O(n)$ with DP array

Summary and Problem-Solving Strategies

Pattern Recognition Guide

Pattern	When to Use	Example Problems
Two Pointers	Sorted arrays, palindromes, pairs	Container With Water, Reverse Vowels
Sliding Window	Contiguous subarrays with constraints	Max Consecutive Ones III
Hash Map	Frequency counting, fast lookup	Two Sum, Group Anagrams
DFS/Recursion	Trees, graphs, backtracking	Max Depth, Path Sum
BFS	Shortest path, level-order	Binary Tree Level Order
Dynamic Programming	Optimization, overlapping subproblems	House Robber, Coin Change
Binary Search	Sorted arrays, search space reduction	Search Insert Position

General Problem-Solving Framework

- 1. Understand the Problem:** Read carefully, identify inputs/outputs, constraints, and edge cases.
- 2. Pattern Recognition:** Does it match a known pattern? Similar to a problem you've solved?
- 3. Plan Approach:** Sketch examples, consider brute force first, then optimize.
- 4. Implement:** Write clean code with good variable names and comments.
- 5. Test:** Run through examples, test edge cases, verify complexity.

Common Edge Cases to Consider

- Empty input (empty array, null tree, empty string)
- Single element (array of size 1, single node tree)
- All same elements
- Already sorted/reversed input
- Maximum/minimum constraints
- Negative numbers and zero

Next Steps

This manual covers detailed solutions for representative problems from each category in LeetCode 75. The patterns and techniques shown here apply to all 75 problems. Practice implementing these solutions, understand the why behind each approach, and you'll develop strong pattern recognition skills essential for technical interviews.

Continue practicing and good luck with your interview preparation!